

LedgerSentinel

Autonomous fraud detection with bounded action. Three owners, one vertical slice, one honest failure fixed on camera.

ASHMIT	YASH	YASHRAJ
Model training	Design + UI	Backend + Supabase + UX

Scope: Phase 0 pre-event preparation plus a 24-hour build. Source spec: LedgerSentinel Full Product PRD & Design Document v1.0.

00 What ships

A transaction that looks clean in isolation. Then a call and an SMS enter evidence, risk rises, confidence falls, and the policy gate takes autonomy away from the AI. The ladder is the product — five rungs now, not four.

RUNG	MEANING
APPROVE	Low risk, high confidence, inside limits. AI acts alone.
VERIFY	Step-up identity challenge on an out-of-band channel.
COOL_OFF	Timed delay. Friction the scammer cannot wait out. (New in v1.1.)
HOLD	Reversible, time-boxed hold. Routed to review.
ESCALATE	Human decides. AI proposes only.

AI autonomy runs from maximum at the top to minimum at the bottom.

New in v1.1: COOL_OFF as its own rung, confidence calibration as a PRISM evaluator, a PII redaction pass before any LLM call, identity-assurance level as an evidence signal, and “fail toward oversight” promoted to a policy rule. Everything else is the PRD as written.

Stack

LAYER	CHOICE	WHY
Frontend	React + Vite + Tailwind	TypeScript, fast HMR through a 24-hour build
Backend	Java 21 + Spring Boot 3	Typed policy gate — an unhandled risk state will not compile
Database	Supabase — Postgres + pgvector	Over JDBC. RLS on every table
On-device	Gemma 3n E4B	Natively multimodal: audio, image, text. Built to run on-device
Server model	Qwen3-4B-Q8	Fusion rationale and advisory synthesis
Model service	Python + FastAPI	Thin wrapper. The only Python in the system

Settle this before Phase 0: the plan assumes Qwen3-4B-Q8 — roughly 4.3 GB, laptop-runnable via Ollama. If you meant Qwen3-32B at Q8 that is ~34 GB and needs a real GPU. Decide now, not at hour 14.

Why Java for the backend

A deliberate change from the PRD’s FastAPI, and it makes the product stronger rather than merely different.

- **The model service was always separate.** It speaks HTTP behind a frozen contract, so the Python ML ecosystem is untouched by the backend language. The polyglot split is clean, not a compromise.
- **The policy gate is the thesis, and it is pure typed business logic.** With sealed interfaces and an exhaustive switch, an unhandled risk state is a compile error. That is a materially better safety story than a Python if-chain — and it demos well.
- **Banks run Java.** For a fraud product, it reads as production-intent rather than prototype.

The trade-off you accept: no LangGraph. Build the agent trajectory as an explicit state machine in Java. The PRD already permits this, and it is more defensible for bounded autonomy — no framework magic, no open-ended loops, hard-typed terminal states you can point at on a slide.

Two Spring gotchas that will cost you hours if ignored:

- **Do not use JPA/Hibernate.** Use Spring Data JDBC or plain JdbcClient. Lazy-loading and entity mapping will eat an afternoon you do not have.
- **The Supabase service role key bypasses RLS.** If Spring connects as service role, RLS is not protecting you. Enforce authorization in the Spring layer and keep RLS as defence in depth — and say exactly that if a judge asks.

01 Two risks to kill before the clock starts

Both are structural. Neither is about skill, and neither fixes itself once the build is underway.

Risk 1 — “Yash does UI” and “Yashraj does UI/UX” is the same file

Two people editing the same components for 24 hours produces merge conflicts at exactly the hour you can least afford them, plus a design that drifts from the comps.

FIX — SPLIT BY LAYER, NOT BY SCREEN

Yash owns presentational components and tokens: markup, CSS, states, responsiveness. He never writes a fetch call. Yashraj owns information architecture, routing, data wiring, and the Supabase/API layer. He never edits a token or a component’s visuals. The seam is a typed props contract plus `fixtures.ts` — Yash builds against fixtures, Yashraj swaps in live data.

Risk 2 — Ashmit alone on two models blocks everyone

If the frontend and backend wait on real inference, nothing is demoable until hour 18 and there is no time left to find a real failure.

FIX — MOCK-FIRST, CONTRACT-FROZEN

Yashraj stands up `/model/voice/score` and `/model/text/score` as mocks returning fixture `Signal[]` in Phase 1. Ashmit ships real inference behind the identical contract. The swap is one env flag. Nobody is ever blocked, and if a model slips you demo with mocks plus Ashmit’s offline metrics.

02 Ownership

Every artifact has exactly one owner. Shared ownership means nobody finishes it.

OWNER	OWNS OUTRIGHT	NEVER TOUCHES
Yash	React + Vite + Tailwind. Design system + tokens, all 8 screens as comps, presentational components, risk/confidence visualisations, empty + error + loading states, the consent screen	Data fetching, Supabase, routing, policy logic

OWNER	OWNS OUTRIGHT	NEVER TOUCHES
Yashraj	Java 21 + Spring Boot 3. Supabase schema + RLS, REST endpoints, the deterministic policy gate, the explicit state machine, evidence fusion, audit log, redaction pass, routing + data wiring, fixtures	Component visuals, tokens, model internals
Ashmit	Gemma 3n E4B + Qwen3-4B-Q8 behind a FastAPI wrapper. Label taxonomy, structured extraction, the calibration layer, eval notebooks, latency + confusion matrix	The policy gate — models emit signals, never decisions
Rotating	PRISM cohort and evaluators. Whoever is least blocked at each gate takes the next evaluator.	—

The one hard rule: models emit `Signal` objects with confidences. Only the deterministic gate emits an action. If Ashmit's model ever returns the string "HOLD", the architecture has been violated and the core thesis of the pitch collapses.

03 Phase 0 — before the clock

The highest-leverage phase, and the one teams skip. You cannot train a model during a 24-hour event. Compress this if you have two days; expand it if you have two weeks — but do not enter the room without the contracts frozen and a baseline model exported.

PHASE 0 T-MINUS • PRE-EVENT

Freeze contracts, train models, draw screens

Goal: on hour zero, three people can work in parallel for eight hours without needing to talk to each other. **Contracts first, in one 90-minute session, all three present.**

All three — 90 minutes, one sitting

- Freeze the JSON for Signal, Evidence, Assessment, Decision. Write them into schemas/ and treat them as immutable.
- Signal = {signal_type, value, confidence, source_ref, evidence_span, redacted_quote}. The span and quote are what the analyst UI shows — agree the shape now or the UI cannot cite evidence later.
- Freeze the action enum at five values and the label taxonomy at eight. No additions after this session.
- Agree the **demo script beat by beat**. You are building toward a five-minute performance, not a feature list.

Ashmit

- **Gemma 3n E4B does all three input types.** Audio, image and text in one multimodal model — it replaces Whisper, OCR and a text classifier. Pull it with `ollama pull gemma3n:e4b` and get structured JSON output working first.
- Write the extraction prompt against the eight frozen labels: urgency, authority impersonation, secrecy request, remote-access request, payment redirect, OTP request, threat, investment lure.
- Multi-label with per-label scores — not binary scam / not-scam. The per-label confidences are the evidence.
- **Fit the calibration layer.** Platt scaling over Gemma’s raw label scores against a labelled holdout. A model writing “confidence: 0.8” is producing a vibe, not a probability.
- Stand up Qwen3-4B-Q8 for server-side fusion rationale. Confirm the quantisation fits your hardware before you commit.
- Assemble data: public scam corpora, SMS spam sets, and synthetic transcripts you generate and hand-check. Log provenance for every row.
- Deliver: FastAPI service on the frozen contract, p95 latency, confusion matrix, calibration curve.

Yash

- Design system in Figma from the PRD tokens — the risk/confidence pair is the hardest widget, solve it first.
- All 8 screens as static comps, plus the consent screen (screen 9, new).
- Component inventory with named states so Yashraj knows exactly what props exist.
- Decide how confidence renders separately from risk. If a judge confuses the two, the pitch has failed.

Yashraj

- Supabase project, schema migrated from the PRD data model, RLS policies on every table.
- Seed 40–60 synthetic transactions, customers, accounts, and the linked communications for all 8 eval scenarios.
- Repo, CI, env management, and the frontend shell that boots.
- **Mock model endpoints returning fixture signals.** This is the single most important deliverable of Phase 0.

EXIT GATE A fake transaction flows end to end on mocks and renders a decision on screen. Schemas are frozen in the repo. Ashmit has a confusion matrix. If this is not true, you are not ready to start the clock.

04 The 24 hours

Seven phases, each closing on a gate that must be merged to `main` and demoed aloud in under two minutes. A gate that cannot be demoed is not passed — cut scope and move on rather than carrying debt into the next phase.

PHASE 1 0–3 H • FOUNDATION

Skeleton that runs

Goal: **every lane is unblocked and provably so**. No feature work until the three lanes can run independently.

Ashmit

- Inference service running locally behind the frozen contract.
- Smoke-test: one call recording and one SMS screenshot in, `Signal[]` out.
- Publish p95 latency to the team — Yashraj needs it for the timeout budget.

Yash

- Tokens into `tailwind.config`, Vite app shell, navigation.
- Build the risk + confidence widget first, against `fixtures.ts`.

Yashraj

- Spring Boot up, `POST /v1/transactions/analyze` returning a hard-coded decision.
- Policy gate as a sealed interface over the five actions, with unit tests — thresholds in config, never in prompts.
- Spring Data JDBC to Supabase. **Not** JPA.
- Audit event written on every decision from the very first commit.

EXIT GATE Frontend renders a decision fetched from the real API. Policy gate has passing unit tests. Ashmit's service answers a curl.

PHASE 2 3–8 H • VERTICAL SLICE

One scenario, all the way through

Goal: scenario `S01` — coercive scam plus large transfer — works end to end on mock signals. **This is the only phase that cannot slip**. Everything after it is enrichment.

Ashmit

- Real Gemma 3n extraction replaces the mock for text. Keep the mock switchable behind `MODELS MOCK`.
- Start the streaming path for calls: chunked audio, sliding window, emit signals as they appear.

Yash

- Transaction detail screen, complete with the evidence list.
- Every signal shows its redacted quote and its source. A score with no factors behind it is a bug.

Yashraj

- Evidence fusion: signals plus transaction features plus identity-assurance level into risk and confidence, computed separately.
- Redaction pass in front of every LLM call — strip account numbers, OTPs, names. Log that it ran.
- Wire the real policy gate. Confirm an unsafe AI proposal gets rejected.

EXIT GATE `S01` runs live: transaction in, communication evidence attached, risk high, confidence stated, gate returns `HOLD + ESCALATE`, case created, audit row written. Demo it aloud. This is your fallback demo for the rest of the event.

PHASE 3 8–13 H • REAL MODELS

Both channels live, retrieval in

Goal: real voice and real text/image inference, plus advisory retrieval with provenance. **Breadth arrives now, on top of a slice that already works.**

Ashmit

- Live-call path working on a recorded sample, then on a live mic.
- Gemma 3n vision on message screenshots — payment confirmations, fake invoices, QR codes. No separate OCR step.
- Calibration curve: predicted confidence against actual correctness on the holdout. Hand the numbers to Yashraj.

Yash

- Communication analysis screen with risk phrases highlighted in place.
- Advisory check screen showing source and matched concepts.

Yashraj

- Vector retrieval over a curated advisory corpus, provenance on every hit.
- **Critical-evidence coverage as a mandatory state transition** — a decision cannot be proposed while retrieved critical evidence is unconsumed.
- Timeouts and the fail-toward-oversight rule: any component failure lowers confidence, never raises autonomy.

EXIT GATE A live call and a screenshot both produce evidence visible in a case. Retrieval cites a real advisory. Kill the retrieval service on purpose and watch the decision degrade toward a human, not toward APPROVE.

PHASE 4 13–17 H • WORKFLOW + CONSENT

Case management, verification, consent

Goal: the analyst workflow closes, and the privacy story becomes something you can **show** rather than assert.

Ashmit

- Harden: batch the eight scenarios through both models, fix the worst failure mode.
- Freeze the model version string. It goes in every decision record.

Yash

- Dashboard, case management, verification screens.
- **The consent screen** — per-channel toggles, plain-language scope, a visible revoke. This screen answers the hardest question in the room.

Yashraj

- Case lifecycle, assignment, notes, audit history endpoint.
- Step-up verification as the VERIFY action; identity-assurance level as a signal feeding confidence.
- COOL_OFF rung with a real timer.
- Consent records in Supabase, enforced in RLS — revocation must actually cut off access, not just hide a button.

EXIT GATE An analyst opens a case, reads the evidence, triggers verification, resolves it, and reads the audit trail. Revoking consent in the UI demonstrably stops communication analysis and pushes the next decision toward escalation.

PHASE 5 17–20 H • PRISM

Run the cohort, find a real failure

Goal: **an honest weakness, discovered by instrumentation, not invented for the slide.** Do not weaken V1 to manufacture one — a real failure is more convincing and you will find one.

All three — this is a joint phase

- Traces on every run: model version, policy version, tool calls, retrieval hits, latency.

- Evaluators: decision correctness, critical-evidence coverage, policy adherence, loop discipline, grounded rationale, escalation safety, latency — and **confidence calibration**, the one that proves the thesis.
- Run all 8 scenarios plus the adversarial set: prompt injection in a message, a poisoned advisory, conflicting high-impact evidence.
- Record V1 numbers before touching anything. Screenshot the trace of the failure you pick.

EXIT GATE V1 scores are written down, and you have one named failure with a trace, a root cause, and a one-line fix you can land in two hours. Pick the failure that best illustrates bounded autonomy, not the easiest one.

PHASE 6 20–22 H • REMEDIATION

Fix it, rerun the identical cohort

Goal: a before-and-after on the **same** scenario IDs. Changing the cohort with the fix invalidates the comparison, and a sharp judge will ask.

Ashmit

- If the fix is model-side: threshold or prompt change, retrain only if it fits the window.
- Rerun and capture V2 metrics.

Yash

- PRISM trace screen — trajectory, tool calls, evaluator results, the V1 vs V2 comparison.
- This screen closes the pitch. Give it the most care.

Yashraj

- Land the fix, usually a policy or state-machine change rather than a model change.
- Rerun with identical scenario IDs and a bumped policy version.

EXIT GATE Side-by-side V1 and V2 on the same cohort, with a measurable improvement and a trace that shows why. Both runs are reproducible from a single command.

PHASE 7 22–24 H • DEMO

Harden and rehearse

Goal: **a five-minute demo that cannot fail**. No new features. None.

Ashmit

- Cache inference for the demo transactions. A cold model on stage is a lost pitch.
- Own the model and calibration questions in Q&A.

Yash

- Visual sweep: empty states, overflow, the projector's contrast.
- Drive the screen during the demo.

Yashraj

- Deterministic fixtures and an offline fallback path. Assume the venue wifi fails.
- Own the architecture, policy, and privacy questions.

All three

- Rehearse three times end to end, timed, out loud.
- Run the Q&A drill in section 06 until the privacy answer is reflex, not recall.

EXIT GATE Three clean timed runs under five minutes, at least one with the network disabled.

05 The two models

Gemma 3n E4B is natively multimodal — audio, image and text in one model — and is built to run on-device. That single choice collapses what would otherwise be three components, and it is what makes the privacy answer in section 06 architecturally true rather than aspirational.

STAGE	HANDLED BY	RUNS WHERE	TRAIN IT?
Call audio → signals	Gemma 3n E4B	On-device	No — prompt for structured output
SMS / email → signals	Gemma 3n E4B	On-device	No
Message images → signals	Gemma 3n E4B (vision)	On-device	No — replaces OCR entirely
Redact	Shared pass: account numbers, OTPs, names, card digits stripped	Before anything downstream	No
Calibrate confidence	Platt scaling on a holdout	On-device, post-hoc	Yes — the only thing you fit
Fusion rationale	Qwen3-4B-Q8	Server	No
Emit	Signal[] with calibrated confidence, source ref, redacted quote	—	—

Calibration is the part that matters. A model writing “confidence: 0.8” into JSON is producing a vibe, not a probability — and the entire “uncertainty reduces autonomy” thesis rests on that number meaning something. Fit a Platt scaling layer over Gemma’s raw label scores against a labelled holdout. It is a few lines of scikit-learn and it is the single most impressive thing you can put in front of a technical judge.

Why multi-label and not a scam score: a single probability cannot be cited as evidence. Eight labelled signals with individual confidences give the analyst something to read, the fusion layer something to weigh, and the calibration evaluator something to check.

06 “So your AI reads all my messages?”

This question is coming, and it decides the room. Do not improvise it. The answer below is thirty seconds, and it works because the architecture genuinely backs each claim — build it this way and the answer is simply true.

REHEARSED ANSWER · ~30 SECONDS · YASHRAJ DELIVERS

“It doesn’t, and the architecture is what stops it rather than a promise.

Nothing is read continuously. The inbox is queried only in a narrow window around a transaction the bank has already flagged. No flagged transaction, no access — there is no background scanning and no bulk copy of your messages anywhere.

On the phone, the classifier runs on-device. What leaves is the signal — coercive language detected, 0.82 — not your message.

The analyst never sees your inbox. They see the extracted claim and one redacted quoted phrase. Raw transcript access needs an elevated role, a written justification, and it writes an audit row you can read. You consent per channel and revoke in one tap.

So it’s consent-based, purpose-limited processing — and the purpose is narrow: stopping you from wiring eight lakh to someone impersonating your bank.”

The three properties the answer rests on

PROPERTY	WHY IT HOLDS
SCOPE Event-triggered pull	Access is a function of a flagged transaction, not a standing subscription. This is an architectural property a judge can verify in your code, which is why it is the first thing you say.
LOCALITY The signal travels, the message stays	Gemma 3n E4B is built to run on-device — this is why the model choice matters. Name it when you answer: the claim is a property of the architecture, not a promise about our intentions.
MINIMISATION Claim and pointer, never the body	The Evidence schema already holds claim, source_ref, confidence. Nothing in it needs the message text. Short retention on evidence, then only the decision record survives.

The follow-ups, and what to say

IF THEY ASK	ANSWER
“What if I revoke access?”	The system degrades to transaction-only. And by our fail-toward-oversight rule, less context means lower confidence, which means more human review — never more autonomy. Revoking makes us more cautious, not blind.
“Who can read the actual transcript?”	Nobody by default. Elevated role, written justification, time-boxed, and it writes an audit row the customer can see. We can audit access as closely as we audit the transaction.
“What about the other person on the call?”	A real two-party-consent question that varies by jurisdiction. In the prototype we analyse the customer’s side and disclose recording. Production needs jurisdiction-specific legal review — we are not going to hand-wave that.
“Isn’t this surveillance creep?”	Creep is prevented structurally: event-triggered access, no bulk store, claim-plus-pointer evidence, hard retention. There is no corpus of your messages for scope to creep into.
“What if the model is wrong and freezes my money?”	It cannot. The model emits signals; a deterministic gate authorises actions. High-impact cases always reach a human, and HOLD is reversible and time-boxed. Being uncertain makes the AI less powerful, not more.
“Why not just detect it from the transaction?”	Because the customer really did authorise it. That is what social engineering is. Transaction-only detection is structurally blind to the exact case that costs the most.

Tactical move: spend fifteen seconds of the demo on the consent screen — scopes, per-channel toggles, revoke. A question you have already answered on screen rarely gets asked adversarially. This is why the consent screen is a Phase 4 deliverable and not a nice-to-have.

07 Cut list, in order

Agree this before hour zero, while nobody is attached to anything. When you fall behind, cut from the top of this list without reopening the debate.

- **Cut first:** the image path. Text and voice alone carry the story.
- **Then:** live microphone capture — demo on a pre-recorded call instead.
- **Then:** the dashboard’s portfolio KPIs. The case screen is what matters.
- **Then:** advisory retrieval, degraded to a small curated lookup with provenance.
- **Then:** COOL_OFF collapses back into HOLD.

- **Never cut:** the vertical slice, the deterministic gate, risk shown separately from confidence, the audit trail, and the PRISM V1→V2 comparison. Those five are the submission.

If you have more than 24 hours: stretch Phase 0 and Phase 5 — better training data and a wider adversarial cohort buy more credibility per hour than any additional screen. Keep Phase 2 and Phase 7 exactly as sized.